Capture the Flag 8: Enroll a New Student

Goal: Extend your multi-screen Student Directory by implementing student enrollment using form fields, controllers, validation, submission, and local data persistence while preserving your existing navigation, student details, and edit modal.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does.
- Understand syntax.
- Explain an error (not fix your error).
- Clarify documentation.

You may NOT ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

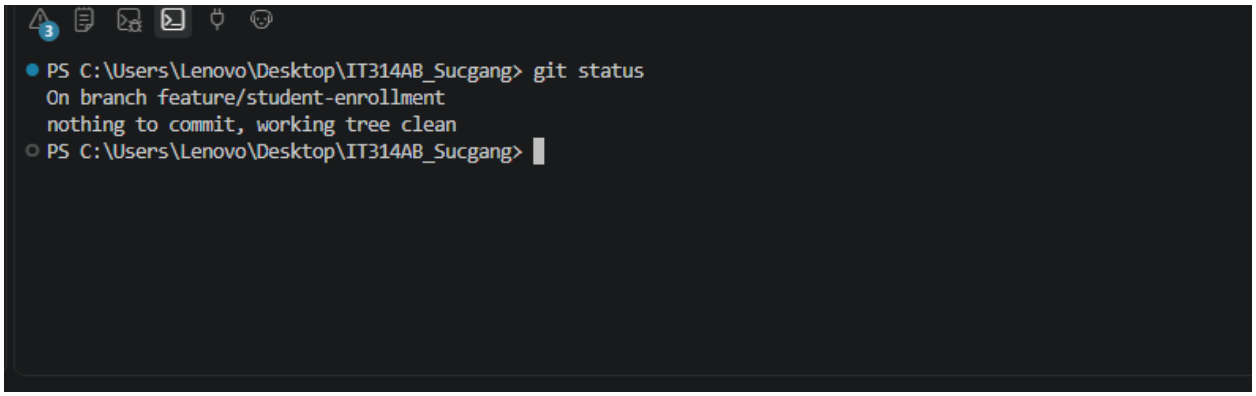
Objective: Create a new feature branch before implementing student enrollment.

Create:

`feature/student-enrollment`

Screenshot:

- Terminal showing your current branch.



🏴 Flag 1 — Build the Add Student Form

Objective: Transform your existing Add Student Screen into a functional enrollment form.

Add the following input fields:

- Student ID
- Full Name
- Course
- Year Level
- Email Address

Research how Flutter uses:

- `TextField` or `TextFormField`
- `TextEditingController`
- Form layouts and input fields

Test

The Add Student Screen should display all required fields and allow administrators to enter student information.

The screen must remain accessible through your existing Student List navigation.

Screenshot:

- Add Student Screen with all input fields.
- Controllers declared in your code.
- Application displaying the enrollment form.




```dart
class _AddStudentPState extends State<AddStudentP> {
  final TextEditingController _studentId = TextEditingController();
  final TextEditingController _fullName = TextEditingController();
  final TextEditingController _course = TextEditingController();
  final TextEditingController _yearLevel = TextEditingController();
  final TextEditingController _emailAddress = TextEditingController();
```

🏴 Flag 2 — Validate Student Information

Objective: Prevent incomplete, invalid, or duplicate student information from being submitted.

Implement form validation with the following requirements:

- All required fields must be filled in.
- Student ID must not duplicate an existing student's ID.
- Email Address must follow a valid email format.
- Year Level must contain an appropriate year level value.

Research how Flutter handles form validation and displays validation messages.

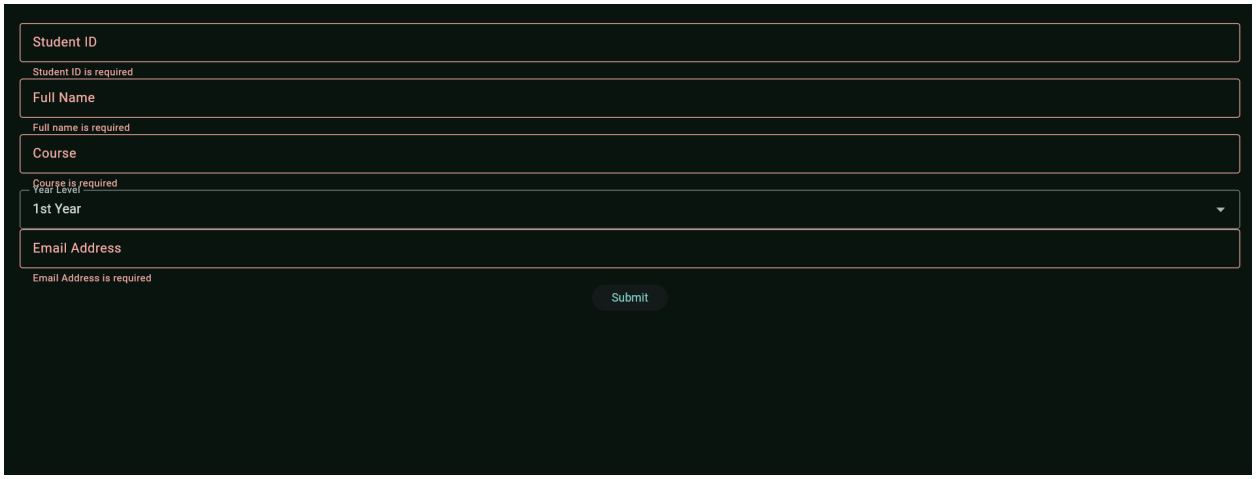
Test

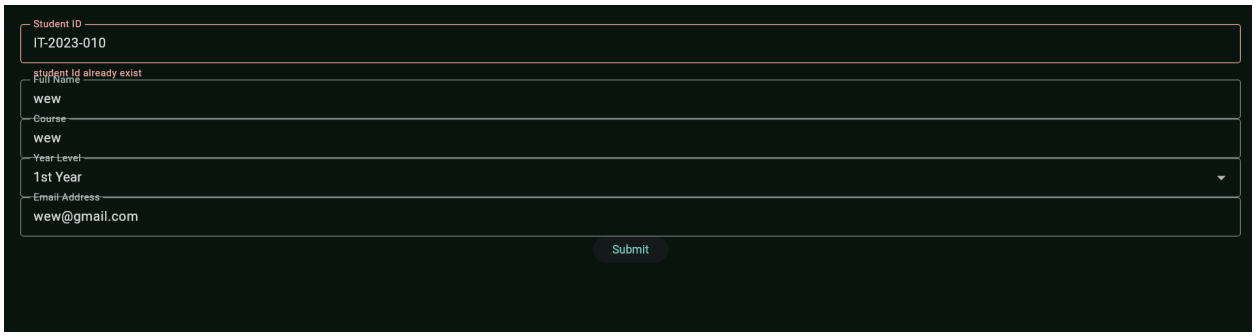
1. Submit the form with all fields empty.
2. Enter an invalid email address.
3. Enter a Student ID that already exists.
4. Enter valid information into every field.

Invalid information should display appropriate validation messages and must not create a student record.

Screenshot:

- Required-field validation.
- Invalid email validation.
- Duplicate Student ID validation.
- Validation logic in your code.





🏴 Flag 3 — Enroll and Display a New Student

Objective: Add a successfully enrolled student to your existing Student Directory.

When valid information is submitted:

- Create a new student record.
- Add the student to your existing student list.
- Return to the Student List Screen or update it appropriately.
- Display the newly enrolled student.
- Clear the form fields after successful submission.

Research how Flutter handles form submission, updating application state, and returning from a screen using navigation.

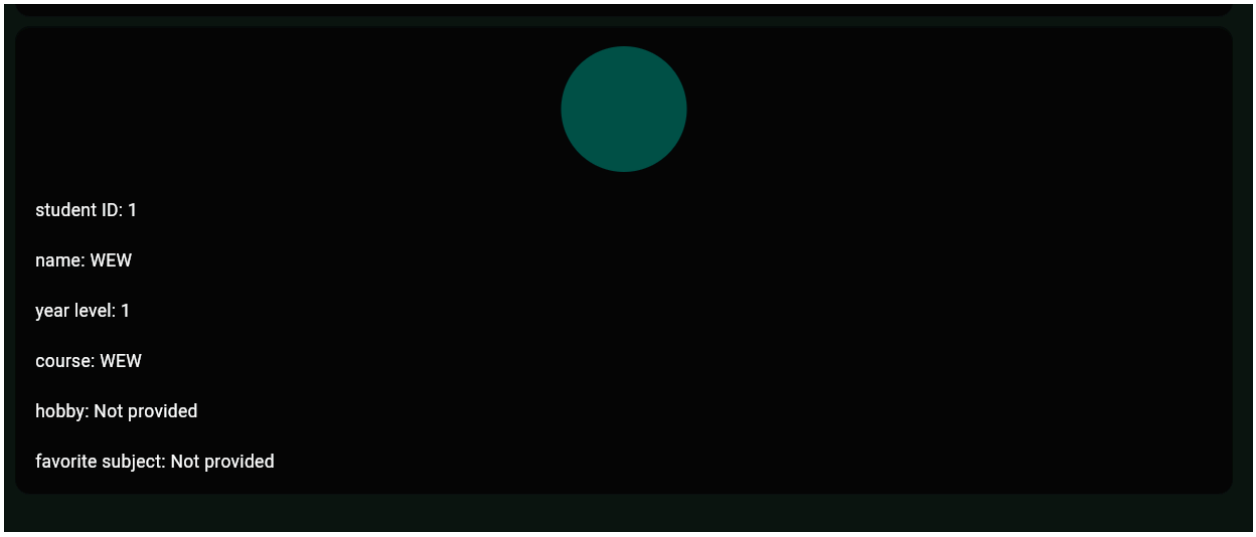
Test

1. Open the Add Student Screen.
2. Enter valid student information.
3. Submit the form.
4. Verify that the new student appears in the Student List.
5. Select the new student.
6. Verify that the Student Details Screen displays the correct information.

Screenshot:

- Completed enrollment form.
- Newly enrolled student in the Student List.
- Student Details Screen displaying the new student's information.
- Submission and navigation logic.





🏴 Flag 4 — Save Student Records Locally

Objective: Preserve student records even after the application is closed and restarted.

Research Flutter-compatible local data persistence options and choose an appropriate method.

Your implementation must:

- Save student records locally.
- Load saved records when the application starts.
- Preserve newly enrolled students after restarting the application.
- Handle situations where no saved records exist yet.

Test

1. Enroll a new student.
2. Close and restart the application.
3. Verify that the newly enrolled student still appears in the Student List.
4. Open the student's details and verify that the information remains correct.

Screenshot:

- Student List displaying saved records.
- Local persistence implementation.
- Student Details Screen displaying a saved student after restarting the application.

```dart
Future<void> _loadStudents() async {
  final preferences = await SharedPreferences.getInstance();
  final savedStudents = preferences.getString(_studentsStorageKey);

  if (savedStudents != null && savedStudents.isNotEmpty) {
    final decodedStudents = jsonDecode(savedStudents);

    if (decodedStudents is! List) {
      throw const FormatException('Saved student records must be a list');
    }

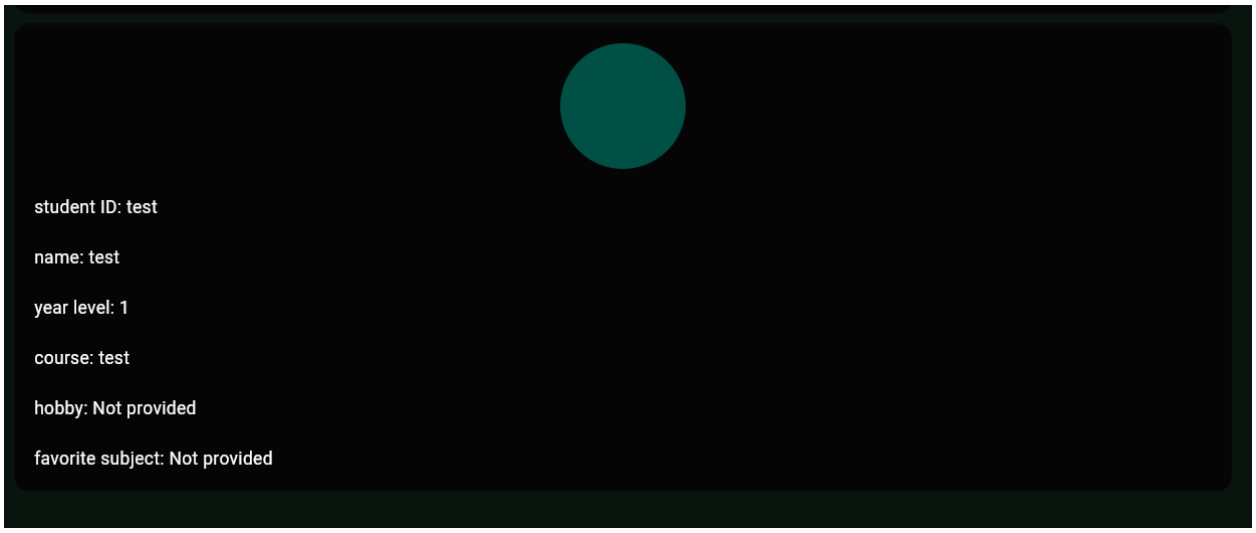
    students = decodedStudents.map((record) {
      if (record is! Map) {
        throw const FormatException('Saved student record is invalid');
      }

      return Student.fromJson(Map<String, dynamic>.from(record));
    }).toList();
  }

  if (mounted) {
    setState(() {
      _isLoading = false;
    });
  }
}

Future<void> _saveStudents() async {
  final preferences = await SharedPreferences.getInstance();
  final saved = await preferences.setString(
    _studentsStorageKey,
    jsonEncode(students.map((student) => student.toJson()).toList()),
  );

  if (!saved) {
    throw StateError('Student records could not be saved');
  }
```



🏴 Flag 5 — Keep Existing Features Functional

Objective: Ensure that enrollment works with the navigation and interactions implemented in CTF 7.

Your application must retain:

- Student List Screen.

- Student Details Screen.
- Student information passed between screens.
- Edit modal.
- Add Student Screen.
- Existing named routes.

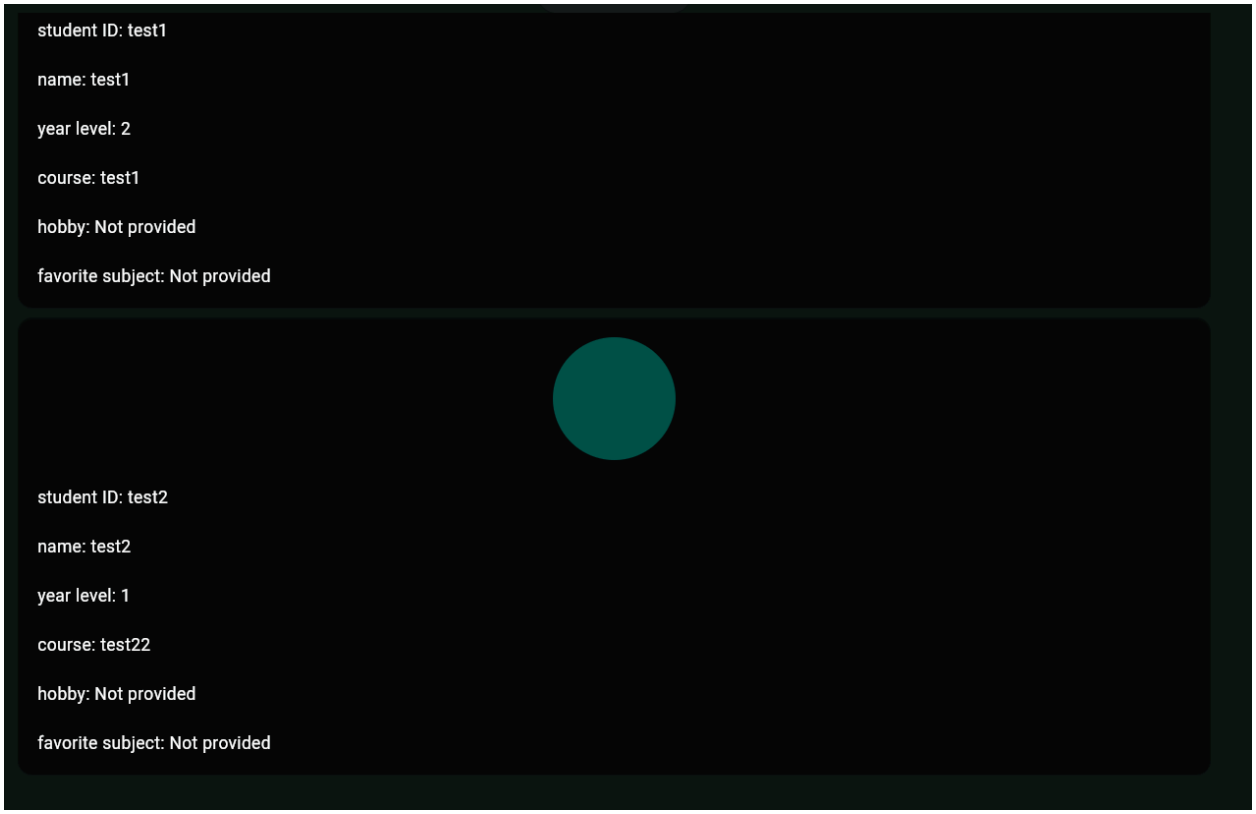
The enrollment workflow must integrate with these existing features rather than replacing them with a single screen.

Test

1. Navigate from Student List to Add Student.
2. Enroll a new student.
3. Open the newly enrolled student's details.
4. Open the Edit modal.
5. Modify the student's information.
6. Close the modal and verify the updated information.
7. Navigate back to the Student List.
8. Verify that the student information remains consistent.

Screenshot:

- Student List with the newly enrolled student.
- Student Details displaying the correct information.
- Edit modal functioning correctly.
- Existing navigation working with the enrollment feature.


student ID: test1
name: test1
year level: 2
course: test1
hobby: Not provided
favorite subject: Not provided
student ID: test2
name: test2
year level: 1
course: test22
hobby: Not provided
favorite subject: Not provided


student ID: IT-2023-001
name: kulafu man





🏴 Flag 6 — Synchronize Changes with Local Data

Objective: Ensure that creating, editing, and deleting students updates the locally saved records.

Your application should keep its student list and locally persisted data consistent.

Verify that:

- Newly enrolled students are saved.
- Edited student information is saved.
- Deleted student records remain deleted after restarting.
- Student IDs remain unique.

- The Student List reflects the latest student records.

Research how to update and remove persisted records using your chosen local storage method.

Test

1. Enroll a new student.
2. Edit the student's information using the modal.
3. Delete a student using your existing delete functionality, if implemented.
4. Restart the application.
5. Verify that the saved records reflect the changes.

Screenshot:

- Student List after enrollment and editing.
- Updated student details.
- Student records after restarting the application.
- Persistence logic for creating, updating, and deleting records.

```dart
Future<void> _saveStudents() async {
  final preferences = await SharedPreferences.getInstance();
  final saved = await preferences.setString(
    _studentsStorageKey,
    jsonEncode(students.map((student) => student.toJson()).toList()),
  );

  if (!saved) {
    throw StateError('Student records could not be saved');
  }
}

void _showSaveError(Object error) {
  if (!mounted) return;

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Student records could not be saved: $error')),
  );
}
```

```dart
Future<void> _updateStudent(
  Student originalStudent,
  Student updatedStudent,
) async {
  final index = students.indexWhere(
    (student) => student.studentId == originalStudent.studentId,
  );

  if (index == -1) return;

  setState(() {
    students[index] = updatedStudent;
  });

  try {
    await _saveStudents();
  } catch (error) {
    if (!mounted) return;
    setState(() {
      students[index] = originalStudent;
    });
    _showSaveError(error);
  }
}
```

```dart
Future<void> _deleteStudent(Student student) async {
  final index = students.indexWhere(
    (currentStudent) => currentStudent.studentId == student.studentId,
  );

  if (index == -1) return;

  final removedStudent = students[index];
  setState(() {
    students.removeAt(index);
  });

  try {
    await _saveStudents();
  } catch (error) {
    if (!mounted) return;
    setState(() {
      students.insert(index, removedStudent);
    });
    _showSaveError(error);
  }
}
```

🏴 Flag 7 — Verify the Complete Enrollment Workflow

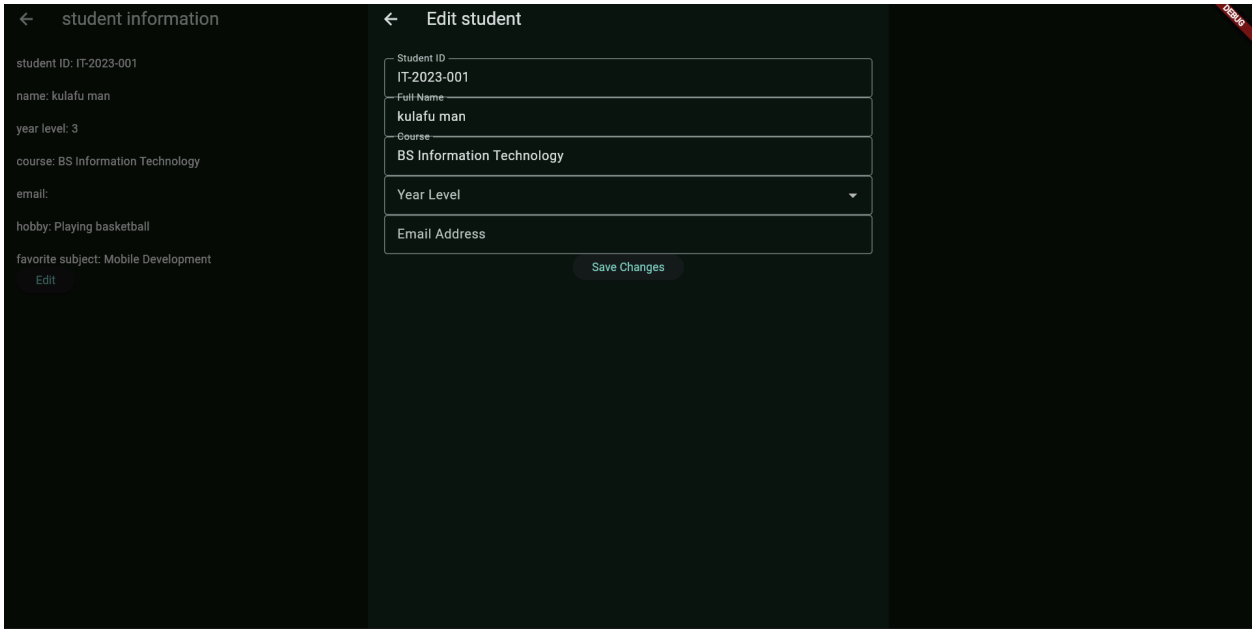
Objective: Demonstrate that the Student Directory supports enrollment, validation, navigation, editing, and local data persistence.

Test Scenario

1. Launch the application.
2. Verify that saved student records load.
3. Navigate to Add Student.
4. Submit an empty form.
5. Submit an invalid email address.
6. Attempt to enroll a student with a duplicate Student ID.
7. Enter valid student information.
8. Submit the form successfully.
9. Verify that the student appears in the Student List.
10. Open Student Details and verify the student's information.
11. Edit the student using the modal.
12. Restart the application.
13. Verify that the enrolled student and updated information remain available.

Screenshot:

- Enrollment form with validation errors.
- Successful enrollment.
- Student List displaying the new student.
- Student Details and Edit modal.
- Saved student information after restarting the application.



🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student Enrollment feature.

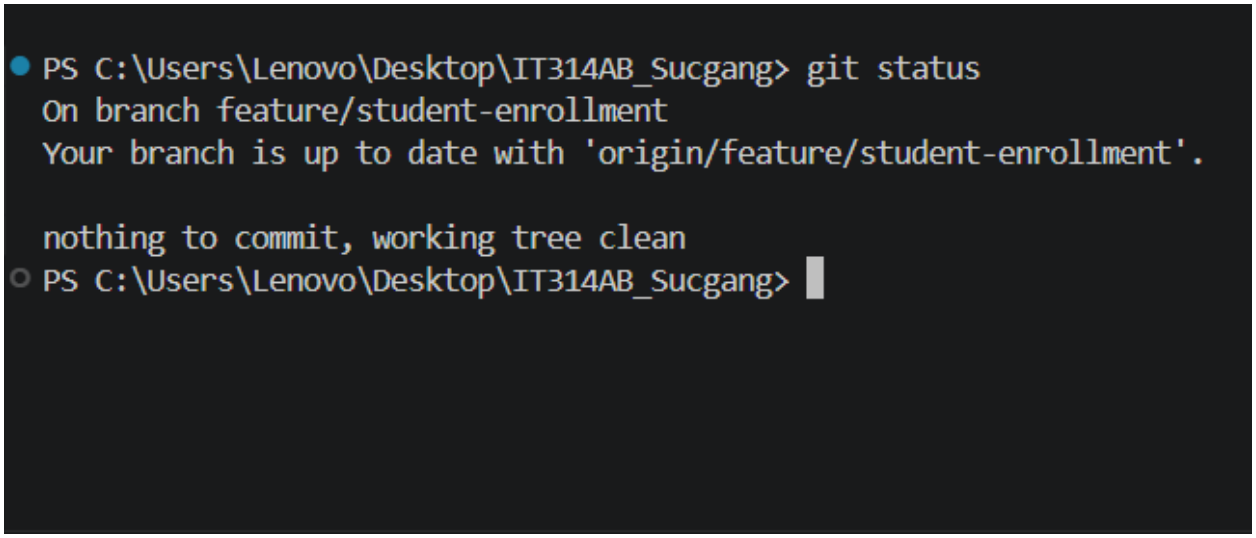
Use a proper Git commit message.

Branch:

`feature/student-enrollment`

Screenshot:

- Successful commit.
- Current branch.
- Terminal showing the commit.
- Successful push to the remote repository.


PS C:\Users\Lenovo\Desktop\IT314AB_Sucgang> git status
On branch feature/student-enrollment
Your branch is up to date with 'origin/feature/student-enrollment'.
nothing to commit, working tree clean
PS C:\Users\Lenovo\Desktop\IT314AB_Sucgang>